



الهيئة الاتحادية للهوية
والجنسية والجمارك وأمن المنافذ
FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

ID Card Toolkit .NET Sample Build Instructions

Version 1.4

March 2026

CONFIDENTIAL

Table of Contents

Revision History	3
Abbreviations	4
1 Introduction	5
1.1 Purpose	5
2 .NET Sample Application Build Instructions	6
2.1 Prerequisites	6
2.2 Build Instructions	6
2.2.1 Step 1: Create a New WPF Project	6
2.2.2 Step 2: Place Sample Source Files	6
2.2.3 Step 3: Add Folder Structure	6
2.2.4 Step 4: Configure App.xaml	6
2.2.5 Step 5: Add Helper Files	7
2.2.6 Step 6: Verify Project Structure	7
2.2.7 Step 7: Add References	7
2.2.8 Step 8: Set Target Framework	7
2.2.9 Step 9: Build the Project	7
2.2.10 Step 10: Set Up Runtime Dependencies	8
2.2.11 Step 11: Run the Sample	8
3 Modern MVVM Sample Application (EIDAToolkitModernApp)	9
3.1 Features	9
3.2 Architecture	9
3.3 Prerequisites	9
3.4 Build Instructions	9
3.4.1 Step 1: Open the Project	9
3.4.2 Step 2: Add Toolkit Reference	10
3.4.3 Step 3: Build	10
3.4.4 Step 4: Set Up Runtime Dependencies	10
3.4.5 Step 5: Run	10

Revision History

Version	Date	Description of Changes
1.0	17th May, 2018	Release Version 1.0.5
1.1	12th Jun, 2018	Release Version 1.0.6
1.2	19th Feb, 2019	Release Version 1.4.5
1.3	20th Dec, 2023	Release Version 3.0.0
1.4	Mar 2026	Toolkit v3.0.5. Updated for Visual Studio 2026 and .NET Framework 4.6.2+. Added modern MVVM sample app build instructions.

Abbreviations

Abbreviation	Description
ICP	Federal Authority for Identity, Citizenship, Customs & Port Security
IDE	Integrated Development Environment
WPF	Windows Presentation Foundation
XAML	Extensible Application Markup Language

1. Introduction

The Federal Authority for Identity, Citizenship, Customs & Port Security (ICP) developed the ID Card Toolkit to address the requirements of service providers (SP) to integrate, into their business applications: Identification, Authentication, Digital Signature and Non-repudiation services, around the capabilities of the Emirates ID Card. The Toolkit is comprised of a number of Software Development Kits (SDK) supporting different programming languages and platforms.

1.1. Purpose

This document is a developer's reference guide to create and build the sample applications provided with the ICP ID Card Toolkit .NET SDK. Two sample applications are included:

- **EIDAToolKitApp** – legacy WPF sample demonstrating core Toolkit functionality.
- **EIDAToolkitModernApp** – modern MVVM WPF sample with a full-featured UI covering all Toolkit services.

2. .NET Sample Application Build Instructions

The following build instructions are based on Visual Studio on Windows. The setup may vary slightly when using different versions of Visual Studio.

2.1. Prerequisites

- **.NET Framework** 4.6.2 or above
- ICP ID Card Toolkit .NET SDK
- **IDE:** Visual Studio 2026
- Windows operating system (Windows 10 or later)

2.2. Build Instructions

2.2.1. Step 1: Create a New WPF Project

1. Open Visual Studio.
2. Go to **File > New > Project**.
3. Select **WPF App (.NET Framework)** from the Visual C# project templates.
4. Set the project name to `EIDAToolkitApp` and click **OK**.

Note: If you use a different project name, update the namespace in all sample source files to match.

2.2.2. Step 2: Place Sample Source Files

1. Right-click the project in Solution Explorer and select **Open Folder in File Explorer**.
2. Copy all sample source files from the `samples` folder of the Toolkit distribution into the project folder. This will replace some of the auto-generated files.

2.2.3. Step 3: Add Folder Structure

Create the following folders in the project and add the corresponding source files:

1. Right-click the project > **Add > New Folder** and name it `Views`.
 - Right-click `Views` > **Add > Existing Item** and select all files from the `Views` folder in the project directory.
2. Right-click the project > **Add > New Folder** and name it `UserControls`.
 - Right-click `UserControls` > **Add > Existing Item** and select all files from the `UserControls` folder.
3. Right-click the project > **Add > New Folder** and name it `Styles`.
 - Right-click `Styles` > **Add > Existing Item** and select `Style.xaml` from the `Styles` folder.

2.2.4. Step 4: Configure App.xaml

Open `App.xaml` and add the following inside the `<Application.Resources>` tag:

```
<Application.Resources>
  <ResourceDictionary>
```

```
<ResourceDictionary.MergedDictionaries>
    <ResourceDictionary Source="Styles/Style.xaml" />
</ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
</Application.Resources>
```

2.2.5. Step 5: Add Helper Files

Right-click the project > **Add > Existing Item** and add the following files from the project directory:

- Helper.cs
- MainWindowHelper.cs

2.2.6. Step 6: Verify Project Structure

After adding all files, the project structure in Solution Explorer should appear as follows:

```
EIDAToolkitApp/
  References/
    IDCardToolkit
    System.Security
    ...
  Styles/
    Style.xaml
  UserControls/
    (user control .xaml and .cs files)
  Views/
    (view .xaml and .cs files)
  App.xaml
  Helper.cs
  MainWindow.xaml
  MainWindowHelper.cs
```

2.2.7. Step 7: Add References

1. Right-click **References** in Solution Explorer > **Add Reference**.
2. Click **Browse** and navigate to the `libs` folder of the Toolkit distribution.
3. Select `IDCardToolkit.dll` and click **Add**.
4. In the same Reference Manager window, go to **Assemblies > Framework**.
5. Check **System.Security** and click **OK**.

2.2.8. Step 8: Set Target Framework

1. Right-click the project > **Properties**.
2. In the **Application** tab, set **Target Framework** to **.NET Framework 4.6.2** or above.

2.2.9. Step 9: Build the Project

1. Select the target platform (**x86** or **x64**) and configuration (**Debug** or **Release**) from the toolbar.
2. Build the project (**Build > Build Solution** or **Ctrl+Shift+B**).

Note: The Toolkit SDK library is protected against debugging. Toolkit API calls cannot be stepped through in the debugger, but your application code around the calls can be debugged normally.

2.2.10. Step 10: Set Up Runtime Dependencies

Place the following files in the same directory as the built executable:

1. **Toolkit library:** Copy `EIDAToolkit.dll` from the `libs` folder of the Toolkit distribution (matching the build architecture – 32-bit or 64-bit).
2. **Plugin libraries:** Copy the required plugin DLLs from the `plugins` folder of the distribution (matching the build architecture).
3. **Configuration file:** Place the application configuration file `config_ap` in the executable directory.
4. **Service Provider configuration:** Place SP-specific configuration files in the subdirectory specified by the `environment` parameter in `config_ap`.

For example, if `config_ap` contains `environment = dev`, create a `dev` subdirectory and place the SP configuration files there.

Note: If `config_ap` is not provided, default application configuration is used and all SP-specific configuration files must be placed directly in the executable directory.

2.2.11. Step 11: Run the Sample

Run the sample project from Visual Studio (**Debug > Start Without Debugging** or `Ctrl+F5`), or execute the built `.exe` directly from the output directory.

3. Modern MVVM Sample Application (EIDAToolkitModernApp)

The modern sample application is a full-featured WPF application built using the MVVM (Model-View-ViewModel) pattern. It provides a comprehensive demonstration of all Toolkit services with a clean, navigable UI.

3.1. Features

The application is organized into 7 navigation pages:

Page	Features
Card Operations	CSN, Card Status, Device ID, License Expiry, Config Certificate Expiry, Interface Type, Register Device, MRZ Parse, Finger Data, Validate Response
Public Data	Non-modifiable Data, Modifiable Data, Home/Work Address, Photo, Signature Image
PIN Operations	Reset PIN, Unblock PIN, PKI Certificates
Biometric Auth	Biometric on Server, Card + Biometric Authentication
Family Book	Head of Family, Wives, Children
NFC Operations	Set/Clear NFC Auth Parameters, Read Public Data via NFC
PKI & Signing	Authenticate PKI, Sign/Verify Data with Auth/Sign Certificate

3.2. Architecture

- **MVVM Pattern:** BaseViewModel, RelayCommand, and async wrappers – no external packages required.
- **ToolkitService:** Wraps the IDCardToolkit.dll P/Invoke layer.
- **Per-view scrolling:** Each page has its own ScrollViewer for independent scroll positions.
- **Status bar:** Single bottom status bar for all operation results.

3.3. Prerequisites

- .NET Framework 4.6.2 or above
- Visual Studio 2026
- ICP ID Card Toolkit .NET SDK

3.4. Build Instructions

3.4.1. Step 1: Open the Project

1. Navigate to the `samples/dotnet/EIDAToolkitModernApp` folder in the Toolkit distribution.
2. Open `EIDAToolkitModernApp.csproj` in Visual Studio.

3.4.2. Step 2: Add Toolkit Reference

1. Right-click **References** in Solution Explorer > **Add Reference**.
2. Click **Browse** and select IDCardToolkit.dll from the `libs` folder of the Toolkit distribution.

3.4.3. Step 3: Build

1. Select the target platform (**x86** or **x64**) and configuration (**Debug** or **Release**).
2. Build the project (**Build > Build Solution** or Ctrl+Shift+B).

3.4.4. Step 4: Set Up Runtime Dependencies

Place the following files in the output directory (same location as the built .exe):

1. **Toolkit library:** EIDAToolkit.dll from the `libs` folder (matching build architecture).
2. **Plugin libraries:** Required plugin DLLs from the `plugins` folder (matching build architecture).
3. **Configuration file:** `config_ap` in the executable directory.
4. **SP configuration:** SP-specific configuration files in the subdirectory specified by the `environment` parameter in `config_ap`.

3.4.5. Step 5: Run

Run from Visual Studio or execute the .exe directly. The application opens with a navigation panel on the left. Select a page to access the corresponding Toolkit features.

Note: The modern sample requires the same runtime dependencies (Toolkit library, plugins, configuration files) as the classic sample. Both samples can coexist in the same Toolkit installation.